CHAPTER 2

Setting Up Terraform

Install Terraform, connect it to AWS, Azure, and GCP, and write your first infrastructure-as-code configuration.

A hands-on walkthrough you can teach from, live

WHAT WE'LL COVER

Four steps from zero to a running EC2 instance

01

Install Terraform

Set it up on Linux, macOS, or Windows

02

Configure providers

Authenticate against AWS, Azure, and GCP

03

Learn HCL & the CLI

The language and commands that drive everything

04

Build & destroy

Launch a real EC2 instance, then tear it down

Terraform runs on every major OS

Pick the path for your platform — the install method differs, but the end result (the terraform CLI on your PATH) is the same.

Linux

Ubuntu / Debian

APT + HashiCorp GPG key

macOS

Homebrew

brew tap hashicorp/tap

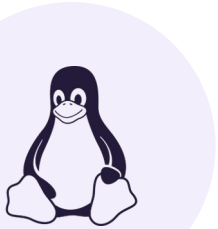
Windows

Manual install

Download binary → add to PATH

Verify any install with `terraform version`

Four commands, one CLI



```
$ sudo apt update && sudo apt install -y \
    gnupg software-properties-common

$ wget -O- https://apt.releases.hashicorp.com/gpg | \
    sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg

$ echo "deb [signed-by=...] https://apt.releases.hashicorp.com \
    $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list

$ sudo apt update && sudo apt install -y terraform
```

Verify: `terraform version`

Two quicker paths

macOS (Homebrew)

```
$ brew tap hashicorp/tap
$ brew install hashicorp/tap/terraform
$ terraform version
```

Windows (manual)

1. Download Terraform from the official HashiCorp site
2. Extract the binary and add it to the system PATH
3. Open Command Prompt and verify the install

```
> terraform version
```

Terraform needs credentials to talk to the cloud

Whichever provider you target, Terraform authenticates using that provider's own CLI and credential files — it doesn't invent its own login system.

AWS

```
aws configure
```

~/.aws/credentials

Azure

```
az login
```

Azure CLI session cache

GCP

```
gcloud auth application-default login
```

Application Default Credentials

Configuring AWS for Terraform

STEP 1 — Install the AWS CLI

```
$ sudo apt install awscli -y   # Ubuntu
$ brew install awscli          # macOS
```

STEP 2 — Run aws configure

```
$ aws configure
```

It will prompt for:

- AWS Access Key
- AWS Secret Key
- Default region (e.g. us-east-1)
- Output format (json recommended)

Stored in ~/.aws/credentials (Linux/macOS) or C:\Users\YourUser\.aws\credentials (Windows)

Rounding out the big three

Azure

Install the Azure CLI:

```
$ curl -sL https://aka.ms/InstallAzureCLIDeb \
    | sudo bash     # Linux
$ brew install azure-cli   # macOS
```

Then log in:

```
$ az login
```

Google Cloud

Install the Google Cloud SDK:

```
$ sudo apt install google-cloud-sdk -y  # Ubuntu
$ brew install google-cloud-sdk         # macOS
```

Then authenticate:

```
$ gcloud auth application-default login
```

HCL: how you describe infrastructure

```
provider "aws" {       # Define the cloud provider
  region = "us-east-1" # Specify the region
}

resource "aws_instance" "example" {  # Define an EC2
instance
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
}
```

Declarative

You describe the end state; Terraform figures out how to get there.

Modular

Infrastructure components are reusable across projects.

Human-readable

Simpler to read and write than raw JSON or YAML.

Six commands you'll use constantly

COMMAND	DESCRIPTION
`terraform init`	Initializes a new or existing Terraform project
`terraform plan`	Shows what Terraform will change before applying
`terraform apply`	Creates or updates infrastructure
`terraform destroy`	Destroys all resources
`terraform fmt`	Formats the configuration files
`terraform validate`	Validates syntax errors in Terraform code

HANDS-ON EXAMPLE

Build a project, then main.tf

Goal: launch a single EC2 instance on AWS using Terraform.

STEP 1 — Create the project directory

```
$ mkdir terraform-ec2 && cd terraform-ec2
$ touch main.tf
```

STEP 2 — Define the EC2 instance in main.tf

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "my_instance" {
  ami           = "ami-0c55b159cbfafe1f0" # Amazon Linux 2 AMI
  instance_type = "t2.micro"

  tags = {
```

HANDS-ON EXAMPLE

From init to teardown

```
terraform init
```

Downloads the AWS provider plugin and creates .terraform

```
terraform plan
```

Previews exactly which resources will be created

```
terraform apply -auto-approve
```

Provisions the EC2 instance and prints its public IP

```
aws ec2 describe-instances ...
```

Confirms MyTerraformInstance is running

```
terraform destroy -auto-approve
```

Deletes everything defined in main.tf — no surprise costs

CHAPTER 2 · RECAP

You now have Terraform up and running

- Installed Terraform on your platform of choice
- Configured AWS credentials
- Learned the core Terraform CLI commands
- Created an EC2 instance using Terraform
- Destroyed the infrastructure cleanly

Up next

Chapter 3 dives into Terraform state — how it's tracked, why it matters, and how to manage it as a team.